

รายงานการตรวจสอบระบบ DRF CRUD API

ระบบบริหารจัดการธุรกรรมร่วมกับ Supabase PostgreSQL & Multi-user Data Isolation

โปรเจกต์:	Django REST Framework CRUD API	ฐานข้อมูล:	Supabase (PostgreSQL)
ระบบยืนยันตัวตน:	SimpleJWT (Bearer Token)	สถานะทดสอบ:	ผ่านการตรวจสอบ (Verified)

1. การรวบรวมชุดโค้ดสำหรับทดสอบ API (JavaScript Fetch)

สคริปต์ JavaScript สำหรับใช้รันทดสอบใน Browser Console เพื่อตรวจสอบการทำงานของ Endpoint ต่างๆ ทั้งการขอ Token และการจัดการข้อมูล Transactions

// 1.1 การขอ Access Token (JWT Request)

```
fetch("http://127.0.0.1:8000/api/token/", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    "username": "user2",
    "password": "Password1234!"
  })
})
.then(response => response.json())
.then(data => console.log("Access Token:", data.access));
```

// 1.2 การดึงรายการธุรกรรมทั้งหมด (GET List)

```
const token = "YOUR_ACCESS_TOKEN";

fetch("http://127.0.0.1:8000/api/transactions/", {
  method: "GET",
  headers: {
    "Authorization": `Bearer ${token}`,
    "Content-Type": "application/json"
  }
})
.then(async response => {
  console.log("Status:", response.status);
  console.log("Data:", await response.json());
});
```

// 1.3 การสร้างรายการธุรกรรมใหม่ (POST Create)

```
const token = "YOUR_ACCESS_TOKEN";

fetch("http://127.0.0.1:8000/api/transactions/", {
  method: "POST",
  headers: {
    "Authorization": `Bearer ${token}`,
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    "amount": "500.00",
    "transaction_type": "Expense",
    "description": "ค่าอุปกรณ์เรียน",
    "transaction_date": "2026-09-09",
    "category_id": 3001
  })
})
.then(async response => {
  console.log("Status:", response.status);
  console.log("Data:", await response.json());
});
```

// 1.4 การจัดการข้อมูลเจาะจงรายไอดี (GET / PATCH / DELETE Detail)

```
const token = "YOUR_ACCESS_TOKEN";
const transactionId = 2;

fetch(`http://127.0.0.1:8000/api/transactions/${transactionId}/`, {
  method: "PATCH", // สามารถเปลี่ยนเป็น "GET" หรือ "DELETE" ได้
  headers: {
    "Authorization": `Bearer ${token}`,
    "Content-Type": "application/json"
  },
  body: JSON.stringify({ "amount": "999999.00" })
})
.then(async response => {
  console.log("Status:", response.status);
  console.log("Data:", await response.text());
});
```

2. สรุปผลการตรวจสอบความปลอดภัยและการแยกสิทธิ์ข้อมูล (Verification Summary)

ผลการทดสอบการเข้าถึงข้อมูลข้ามผู้ใช้ (Cross-user Access Attack/Test) โดยการใช้ JWT Access Token ของ user2 พยายามเรียกดู แก้ไข และลบข้อมูลธุรกรรม ID: 2 ซึ่งเป็นของ user1

HTTP Method	เป้าหมายการทดสอบ	Status Code	คำอธิบายผลการตรวจสอบ
GET (List)	ดึงรายการธุรกรรมทั้งหมดของ User2	200 OK	คืนค่าอาร์เรย์ว่าง [] ไม่พบรายการธุรกรรมของ User1 ปะปนมา

HTTP Method	เป้าหมายการทดสอบ	Status Code	คำอธิบายผลการตรวจสอบ
GET (Detail)	พยายามแอดดู Transaction ID: 2	404 Not Found	ระบบแจ้งว่าไม่พบข้อมูล ป้องกันการสุ่มเดา ID ข้อมูลผู้ใช้อื่น
PATCH	พยายามแก้ไขยอดเงินใน ID: 2	404 Not Found	ปฏิเสธการแก้ไข ไม่อนุญาตให้เปลี่ยนข้อมูลข้ามผู้ใช้
DELETE	พยายามลบข้อมูล ID: 2	404 Not Found	ปฏิเสธการลบ ข้อมูลของ User1 ปลอดภัยจากการถูกลบโดยผู้อื่น

3. อธิบายการทำงานของระบบ (System Architecture & Security Mechanics)

1. ชั้นยืนยันตัวตน (Authentication Layer - SimpleJWT)

เมื่อ Client ส่ง Request พร้อม Header Authorization: Bearer <TOKEN>มายัง API ระบบ Django REST Framework จะทำการ Decode และ Verify ความถูกต้องของ JWT Token หาก Token ถูกต้อง สมบูรณ์ และยังไม่หมดอายุ ระบบจะดึงค่าตัวตนผู้ใช้นั้นๆ ไว้ใน request.user โดยอัตโนมัติ

2. ชั้นแยกสิทธิ์ข้อมูลระดับ Row (Data Isolation Layer - Queryset Filtering)

ใน ViewSet มีการOverride Method get_queryset() เพื่อกรองข้อมูลจากฐานข้อมูล Supabase ตั้งแต่ระดับ Query Level:

```
def get_queryset(self):  
    return Transaction.objects.filter(user=self.request.user)
```

ทำให้ทุกๆ Operation (GET, PUT, PATCH, DELETE) ที่เรียกผ่าน ViewSet จะถูกจำกัดขอบเขต (Scope) ไว้เฉพาะข้อมูลที่ผู้ใช้รายนั้นเป็นเจ้าของเท่านั้น

3. พฤติกรรมตอบกลับเพื่อความปลอดภัย (Security Response Behavior)

เมื่อผู้ใช้พยายามเข้าถึงหรือแก้ไข Resource ID ที่ไม่ได้อยู่ใน Queryset ของตนเอง ระบบจะค้นหา Object ไม่พบและตอบกลับด้วย 404 Not Found แทนที่จะเป็น 403 Forbidden ซึ่งเป็นไปตาม Best Practice ด้าน Security เพื่อไม่ให้ผู้ไม่หวังดีรู้ว่า ID นั้นมีตัวตนอยู่ในระบบหรือไม่